UML Use Case Specification: Sudoku Puzzle Solver (By Gemini)

a. Identify/Discover a Use Case

- **Use Case Name:** Solve Sudoku Puzzles
- **Primary Actor:** User / Student
- **Supporting Actor:** File System / Sudoku Puzzle Files (puzzle1.txt through puzzle5.txt)

b. Brief Description for the Use Case

The user initiates the object-oriented program to load, process, and solve multiple partially completed 9x9 Sudoku puzzles stored in text files using a recursive depth-first search with backtracking, subsequently displaying the original puzzles and their corresponding solutions.

c. Step-by-Step Flow (Outline) of Events for the Use Case

1. **[Actor]** The user compiles and executes the program on the EECS cycle servers.
2. **[System]** The program initializes an array of puzzle filenames (puzzle1.txt through puzzle5.txt).
3. **[System]** For each file entry, a SudokuBoard object is instantiated.
4. **[System]** The SudokuBoard attempts to open and read the puzzle grid from the text file.
5. **[System]** The program prints the filename and the initial grid configuration to the standard output.
6. **[System]** The recursive backtracking algorithm scans for empty cells (_), validates candidate digits (1–9) against row, column, and 3x3 block constraints, and recursively builds solutions.
7. **[System]** The program outputs all discovered solutions for the current puzzle file (or displays "No solution found" if none exist).
8. **[System]** The loop proceeds until all five puzzle files are processed, terminating execution successfully.

d. Detailed Description for the Use Case

- **Preconditions:**
    - The five puzzle text files (puzzle1.txt through puzzle5.txt) must be present in the working directory.
    - The C++ compiler must successfully compile Assignment4.cpp into Assignment4.exe on the cycle server.
- **Postconditions:**

    - All valid puzzle solutions are completely printed to the console output.
    - Program terminates with a return code of 0.
- **Main Success Scenario:**
    - The user executes the compiled program binary.
    - The system iterates through the five puzzle files sequentially.
    - Each file stream successfully populates the 9x9 char grid.
    - The object-oriented solver successfully executes depth-first search with backtracking, finding and printing all valid grid configurations.
- **Alternate / Exception Flows:**
    - *File Open Failure:* If a puzzle file cannot be opened, the system catches the error, outputs an error message for that specific file, and proceeds to the next puzzle.
    - *No Solution Exists:* If a puzzle yields no valid states under constraints, the system prints "No solution found" for that puzzle block.